Nexus Loop — Solution Design

An autonomous improvement loop for AI agent deployments · TechQuest submission · Yellow.ai problem statement

Executive Summary

We build a pipeline that reads eight weeks of agent logs and closes the Detect → Diagnose → Prescribe → Verify loop with exactly one human touchpoint: the approve/reject decision. The system is deterministic-first (facts come from logs, never from a model), baseline-plural (three baselines, because each planted problem defeats a different naive comparison), and refusal-native (it treats 'this cannot be measured' as a first-class, specified output rather than a failure).

Our scoring strategy follows the problem statement's own arithmetic: a Level-1 report with an excellent screen and decision layer scores ~70/100, while a Level-3 report with neither scores ~58. We therefore optimise depth on one problem plus product quality before breadth: nail one real problem end-to-end (finding → impact → diagnosis → fix → replay proof → recorded human decision), dismiss all three lookalikes in writing, refuse A11 with a usable event spec — then expand to the remaining problems only with time left over.

Design Tenets

- **Logs are ground truth; models judge meaning only.** API errors, model versions, handoffs are read from execution steps. An LLM judge is used solely for resolution/satisfaction/topic, is versioned, and publishes its agreement with the 400 human labels (Rule 1).
- **The yardstick is frozen.** Metric definitions, judge versions, and the reviewed good-conversation set are configuration constants, hashed at startup; a fix can never touch them (Rule 2).
- **Every number carries its passport.** Fidelity (measured / judged / derived), coverage (share of traffic that can even produce it), and calibration for judged numbers are emitted at compute time, not annotated later.
- **A finding is a decision.** Every finding is rendered as: affected conversations and traffic share, user outcome delta, duration and cost, owner, consequence of inaction, and a check-able derivation with the baseline named.
- **Unattended by construction.** One entrypoint (run_loop.py) with zero interactive input, so the day-6 sealed run is just the same command on a different corpus path.

1 · Reading the Problem: What Actually Scores

The rubric splits 55 machine-scored points (diagnostic accuracy 20, specificity 15, honesty 12, loop completeness 8) and 45 human-scored points (decision quality 18, screen 15, framing 12). Three consequences drive the architecture:

- **Specificity is worth almost as much as detection.** Flagging a lookalike as a regression costs more than missing a real problem. So the detector is paired with a dismissal engine that must produce a written verdict for every candidate anomaly — nothing is reported without surviving explicit lookalike tests.
- **Honesty points are cheap and deterministic.** Fidelity/coverage on every metric, a calibration that is not 1.00, and the A11 refusal are all mechanical once the catalog is modelled. We bank these on day 1–2.
- **Half the human points are about legibility, not analysis.** The screen and the framing reward a stranger being able to act. The report generator and the screen share one data model so evidence on screen is always traceable to the JSON.

We also take the kit's own hints literally: run starter.py before writing code and copy its coverage-handling pattern; read catalog.json end-to-end before any query (the safe-join map is what defuses the denominator traps); take A11 first because it is the cheapest real scoring; and distrust everything that ‘stands out’ in the first grouping, since three of those spikes are planted decoys.

2 · System Architecture

Plain Python, no dependencies (matching the kit), one directed pipeline. Each stage writes an intermediate JSON artefact so any figure in the final report can be traced backwards to raw log lines.

Stage	Component	What it does
0	Ingest	Streams gzipped JSONL (conversations, execution steps, transcripts, config changes, ratings) into normalised per-conversation records. Validates every field against catalog.json; unknown or unjoinable fields are quarantined, not guessed.
1	Metric layer	Computes metrics as (value, fidelity, coverage, calibration) tuples. Coverage is computed from the denominator that can produce the metric — e.g. tool-error rate over only the traffic that logs tool calls.
2	Detect	Segments traffic intent × tenant × week; runs each segment against three baselines (§3). Emits candidate anomalies with effect sizes, never conclusions.
3	Diagnose	Aligns each surviving anomaly to the 16 timestamped config changes (changepoint vs change-time proximity + affected-slice match), producing a blamed change or an explicit ‘no config candidate’.
4	Dismiss	Runs every candidate through lookalike tests (§4). Dismissals are written into findings as ‘decided not a problem’, with the discriminating evidence.
5	Refuse	Measurability engine: answers the 11 operator asks from the catalog model, or emits a gap with the exact event schema that would need to exist (§5).
6	Prescribe	Mines the standard from the deployment's own best conversations; drafts a fix as a decision: what to approve, risk if the diagnosis is wrong, and a no-ship condition.
7	Verify	Replays the fix on the affected cohort via replay/serve.py, before/after on the frozen metrics. A null effect is recorded as a valid negative result.
8	Decide	The screen renders one finding + evidence + approve/reject; the human decision is written back into loop-report.json. A hit-rate ledger accumulates system self-assessment.

3 · Detection Methodology: Three Baselines, Because One Is Never Enough

The brief states each planted problem defeats a lazy approach: one has no before-period, one is invisible in error rate, one never moves the outcome metric. A single anomaly detector therefore cannot find all three. We run three baseline families over every intent × tenant × week segment and require a candidate to name which baseline exposed it.

3.1 Temporal self-baseline (before vs after)

Classic changepoint comparison: each segment's metric trajectory against its own history, aligned to the 16 config-change timestamps. Catches regressions with a before-period — e.g. an API that begins returning empty payloads while reporting HTTP success. Because the obvious error-rate metric is blind to it, we detect it on *behavioural* signals read directly from logs: response-payload size distribution per tool, retry/re-ask density, and downstream resolution of conversations that touched that tool.

3.2 Peer baseline (segment vs comparable segments)

For the problem with no before-period — a new product launched with nothing in the knowledge base — its traffic slice is new, so self-history does not exist. Its baseline must be peer traffic: same tenant, other intents; and same intent-family, other tenants. Signature we expect: KB-retrieval steps returning zero/low-relevance hits, elevated human-handoff and unresolved rates versus peers, onset at the launch date rather than at any config change.

3.3 Best-of-deployment baseline (the mined standard)

For the problem that never moves the outcome metric — a prompt change that doubles conversation length with unchanged outcomes — outcome deltas are zero by construction. It is only visible against the deployment's own best conversations: same intents resolved in half the turns/tokens before the change. We mine the standard (Task 02) from top conversations selected by deterministic outcome facts (resolved without handoff, no tool errors, few turns) plus sparse human ratings — never by our own judge, to avoid circularity — and express it as per-intent envelopes: turns, tokens, tool-call count, handoff rate. Deviation from the envelope with flat outcomes is the cost finding: same results, roughly double the tokens and user time; the impact block prices it in token-cost and user-minutes.

3.4 Attribution discipline

A candidate becomes a finding only when (a) onset aligns to a config change or an external, verifiable event (product launch), (b) the affected slice matches the blast radius that change could plausibly have, and (c) it survives the dismissal battery below. Findings state detection latency — how many days the problem ran before our system would have flagged it — since the rubric scores ‘how late you noticed’.

4 · Dismissing the Lookalikes, in Writing

Lookalike	Why it tempts	Discriminating test → written dismissal
Marketing campaign shifts question mix	Aggregate resolution moves	Decompose: per-intent resolution stable; only the intent *mix* changed. Mix-adjusted (weighted) resolution shows no regression. Verdict: distribution shift, not behaviour change — no config change aligns.
Flash sale spikes traffic + latency, 4 days	Latency and volume alarms	Latency rises with volume and fully reverts when volume does; resolution and handoff within normal band throughout; no config change in window. Verdict: capacity event, self-resolved.

Lookalike	Why it tempts	Discriminating test → written dismissal
Quality rubric version change mid-corpus	Judged quality scores jump at the boundary	Discontinuity is exactly at the rubric version timestamp; scores computed within each rubric version are flat; deterministic outcome facts (resolution, handoffs) show no step. Verdict: the yardstick moved, not the agent — and per Rule 2 we never mix scores across judge versions.

Each dismissal is emitted into the findings section as a ‘not a problem’ entry carrying the same evidence discipline as a positive finding. This is where the 15 specificity points live, and it is also the honest thing to do: an operator who sees the spike deserves to know why we ignored it.

5 · Refusals and Denominator Honesty

5.1 The unanswerable question (A11, failover rate)

The measurability engine walks each operator ask against the catalog: which event type would carry the answer, and does it exist? For failover rate the runtime has no failover mechanism, so no event of that kind is ever logged. The refusal is specified, not shrugged:

```
GAP-A11 · Question: 'What is our failover rate?' · Status: unanswerable from this
deployment's logs. · Missing event: an execution-step record of type failover_triggered
with fields {conversation_id, timestamp, from_target, to_target, trigger_reason,
success}. If the runtime emitted this event, the metric would be
count(failover_triggered)/count(eligible_requests), with coverage bounded by the
deployment date of that instrumentation. Until then, any number offered for this
question would be invented.
```

5.2 Denominator trap 1 — legacy traffic logs no tool calls

Any tool metric's denominator is only the traffic that records tool calls. We compute that share from the data (per tenant, per week) and stamp it as the metric's coverage — e.g. 'tool-error rate: fidelity=measured, coverage=NN% of conversations (legacy pipeline logs no execution steps)'. We never silently extrapolate tool behaviour to legacy traffic, and trends are reported over covered traffic only, with the covered share shown alongside so a mix change cannot masquerade as a behaviour change.

5.3 Denominator trap 2 — the near-unique field

A field that is nearly unique per customer invites a per-value breakdown with n≈1 cells — statistically meaningless and a privacy smell. The metric layer enforces a cardinality budget: any grouping key whose distinct-value count exceeds a threshold relative to row count is refused as a breakdown dimension, and the refusal is written into the report citing the budget ('field X: ~1 value per customer; a breakdown would have a median cell size of 1 and cannot support any claim'). It remains usable as a join key where the catalog marks the join safe.

5.4 Rule 1 / Rule 2 compliance and the judge

- **Deterministic facts:** errors, model versions, handoffs, tool calls, timestamps — read from logs. Any code path that would ask a model for these fails a lint we run in CI.
- **Judge:** one versioned resolution/quality judge, prompt + version hash recorded in every judged number. Calibrated against the 400 human resolution labels and ~100 quality scores; we publish raw agreement. Human labellers disagree ~7.5% of the time, so agreement is expected to plateau around ~0.92–0.93 — a 1.00 would be flagged by our own self-check as a leak or a bug.
- **Frozen yardstick:** metric definitions, judge version, and the reviewed good set are loaded from a config whose hash is printed in the report header; the prescribe stage has no write path to it.

6 · Fix Proposal and Replay Verification

A fix is framed as a decision, not a patch: **what** we ask a human to approve (e.g. 'add the missing product's KB articles / revert prompt to version N-1 for the affected intents'), **the risk if the diagnosis is wrong** (e.g. the verbosity is actually driven by a traffic-mix change and reverting loses an unrelated improvement), and **the no-ship condition** (e.g. replay shows no improvement on the affected cohort, or any regression on unaffected cohorts).

Replay budget — 40 attempts, spent like money:

- Reserve: 8 attempts held back for the sealed-run week and demo-day contingency.

- Calibrate: 4 attempts on a null fix and on deliberately-wrong fixes to learn the service's variance and confirm that ‘no effect’ is readable.
- Iterate: up to 20 attempts, always on the affected cohort plus one control cohort, one variable changed per attempt.
- Confirm: final before/after for each shipped finding on the full affected slice; these are the numbers that enter the report.

A wrong fix returning no effect is recorded as a verification result with verdict ‘no effect — diagnosis or fix rejected’, feeding the self-assessment hit-rate rather than being retried until something moves.

7 · The Screen: One Finding, Evidence, a Decision

One screen, three zones, zero navigation. It is generated from the same JSON the scorer reads, so nothing on screen can disagree with the report.

Zone	Content	Monday-morning test
Finding	Plain-language headline; impact block: conversations affected and share of traffic, users who would have resolved and did not, users who reached a human who was never meant to be involved, days running, cost to date; owner (agent builder / business owner / platform); one-sentence consequence of inaction.	An operator who was not on the team knows what broke, how much it hurts, and whose job it is.
Evidence	The two-line derivation with the baseline named; a before/after or peer-comparison chart; every number annotated with fidelity · coverage · calibration on hover/inline; the dismissed lookalike nearest to this finding, with its verdict; the refused question, with its event spec.	Every figure is check-able against the data; the definitions travel with the numbers.
Decision	The proposed fix as a decision (approve what, risk, no-ship condition), replay before/after on the affected cohort, and a working APPROVE / REJECT control with a required reason field. The choice is written back into loop-report.json and into the hit-rate ledger.	A rejection with a good reason scores as well as an approval — the control makes rejection a first-class path, not a failure state.

Implementation: a single static HTML page reading loop-report.json, plus a ~50-line stdlib HTTP endpoint that appends the decision back into the report. No framework, nothing to break during the sealed run.

8 · Surviving the Sealed Run

• **No memorised answers.** Detection thresholds, cohort definitions and config-change alignment are computed from whatever corpus is mounted — no day numbers, tenant names or intent strings hardcoded from practice data. We grep the codebase for practice-data literals before the freeze.

• **One command, no hands.** `python run_loop.py --corpus PATH` produces the full report; the screen renders whatever report exists. A system that needs hand-holding scores as if it failed, so the practice week's daily ritual is running the loop cold from a clean checkout.

• **Degrade honestly.** If a stage finds nothing (e.g. no verifiable fix in budget), it emits an empty-but-valid section with a stated reason, keeping the report schema-valid and the honesty fields intact rather than inventing content.

• **Self-scoring cadence.** score.py runs in CI against every commit from day 2; the practice ground truth is used only for scoring, never as a feature.

9 · Execution Plan (7 people, 6 days)

Day	Engineering track (4)	Product track (3)	Exit test
1	make kit; starter.py; score.py. Model catalog.json; write GAP-A11; one metric with fidelity+coverage.	Read asks.md as operators; draft finding/decision block language from the Level examples.	On the board; A11 refusal drafted; one honest number on screen.
2	Valid report emitted; metric layer done; standard mining starts; sample-vs-full parity check.	Screen skeleton bound to loop-report.json.	score.py returns a number from our own report.

Day	Engineering track (4)	Product track (3)	Exit test
3	Three-baseline detection on the sample; dismissal battery; confirm on full corpus; diagnosis→config alignment.	Evidence zone with real figures; dismissal rendering.	Level-1 bar: one real problem quantified, three lookalikes dismissed, A11 refused — self-scored ≈38/55.
4	One fix proposed; replay calibration + iteration on affected cohort.	Decision zone; approve/reject writes back.	Before/after on the affected cohort in the report.
5	Cold-run drills from clean checkout; remaining problems if budget allows; hit-rate ledger.	Framing rehearsal: open on the problem, refusal out loud, state coverage, admit what is faked.	Full loop on one problem with no human except the approval.
6	12:00 freeze; sealed run untouched.	15:00 demo: 10 minutes, operator's side first.	Sealed report produced without edits.

10 · Tradeoffs — What We Chose and What It Costs

Decision	Chosen over	Why, and the cost we accept
Depth on one problem + product polish before breadth	Chasing all three problems early	The PS's own arithmetic: Level 1 + great screen ≈ 70 > Level 3 + weak product ≈ 58. Cost: we may present only 1–2 verified fixes; remaining problems appear as detected-but-unverified findings.
Deterministic detectors; judge only for meaning	LLM-heavy anomaly narration	Rule 1 is disqualifying if broken, and deterministic signals are reproducible on the sealed run. Cost: subtle semantic regressions outside our behavioural signals could slip past; the judge's coverage is bounded by calibration-label volume.
Three explicit baseline families	One general anomaly detector	Each planted problem defeats one naive baseline, and named baselines make derivations check-able (rubric: ‘baseline named’). Cost: three code paths to test in six days; mitigated by sharing the segmentation layer.
Conservative reporting threshold (dismiss when in doubt)	High recall	A false alarm on a lookalike costs more than a miss, per the PS. Cost: a borderline real problem might be reported as ‘candidate, not confirmed’ rather than a finding — we surface it in Gaps rather than overclaim.
Standard mined from deterministic outcomes + human ratings	Mining with our own judge	Avoids circularity (judge defines good, then measures against it) and keeps the reviewed good set independent of anything we could tune (Rule 2). Cost: sparse ratings shrink the mined set; envelopes are per-intent-family rather than per-intent where data is thin, and that granularity is stated.
Plain Python, static HTML, stdlib server	Pandas/React stack	Matches the kit, nothing to install, nothing to break unattended at 12:00 on day 6. Cost: slower ergonomics for us and a plainer screen; we spend the saved time on content and framing, which is where the 45 human points are.
Replay budget with an 8-attempt reserve	Free iteration	40 attempts is the scarcest resource; running out before the confirming run would leave a fix unproven. Cost: fewer exploratory replays; we compensate with offline cohort analysis before spending an attempt.
Streaming single-pass ingest, 5% sample for iteration	Loading 80k conversations in memory	Keeps the loop fast enough to run many times a day and honest about the sample-vs-full step the kit mandates. Cost: two-phase workflow; every sample result must re-confirm on the full corpus before entering the report.

11 · Assumptions Note: Real, Stubbed, and What Breaks at 100×

What is real

• Ingest, metric layer with honesty fields, three-baseline detection, lookalike dismissals, config-change attribution, the A11 gap spec, coverage/cardinality refusals, report generation, schema validation, and the approve/reject write-back.

• Replay verification numbers — every before/after shown comes from replay/serve.py, not from our own simulation.

What is stubbed or simulated

• The judge is a single versioned prompt over one model, not an ensemble; its calibration is reported, not optimised.

• The hit-rate ledger self-assessment is a simple running tally of approved / rejected / no-effect verdicts — a record, not a learning system.

• Cost figures use a stated per-token and per-human-handoff price constant, declared in the report, since real unit economics are not in the corpus.

• Alerting/scheduling is out of scope: the loop runs on demand, not as a resident service.

What breaks at 100× scale

• 8M conversations / 88M steps: single-machine streaming passes become hours; the fix is partitioned ingest (per tenant/week shards) and pre-aggregated metric cubes — the metric contracts already assume aggregation, so the honesty fields survive the rewrite.

• Judge cost: judging every conversation is unaffordable; we would judge a stratified sample per segment and report the sampling coverage — the coverage field is exactly where that honesty lands.

• The append-to-JSON decision write-back needs a real store with audit history once multiple operators decide concurrently.

• Segment explosion (intents × tenants × weeks) requires interest-ranked evaluation rather than exhaustive scanning; detection latency guarantees would need to be restated per tier of traffic volume.

One line version: the deployment already writes down everything it does. This system is the reader it never had — one that knows the difference between a problem, a coincidence, and a question the data cannot answer, and that ends every loop with a human saying yes or no.